

16.5

CS5617- SOFTWARE ENGINEERING 2026: QUIZ 1 [Max: 24 marks]

Name: M. Murali Karthick

Roll number: 112301019

Question 1 (1 marks)

What is the difference between method overriding and method overloading in C#?

Method override is used ^{by child class} to change the current implementation by parent class.

Method overloading is when a single method behaves differently in different classes.

Question 2 (1 marks)

What is the purpose of the virtual and override keywords in C#?

In C#, generally the virtual keyword is written ahead of methods define in parent class, ~~for~~ which along with override keyword used in child class can overwrik the parent implimentation, to some ~~the~~ function that child desire. ~~It~~ In short it is used for method ~~overing~~ overriding.

Question 3 (1 marks)

Name any two SDLC models. Which model are you going to use for your group project?

- Agile.
- Waterfall.

We are going to use the Waterfall SDLC model.

Question 4 (1 marks)

A C# application must use a legacy temperature sensor that reports Fahrenheit, while the application expects Celsius. Name the most suitable design pattern, its category, and the role it plays.

We use Adapter design pattern, It is ~~by~~ a structural design patten. The adapter ~~class~~ plays the role of conversion by taking the reading in Fahrenheit and converting to Celcius for application to use.

Question 5 (1.5 marks)

What does a UML module diagram show? State two rules for arranging dependencies between upper-level and lower-level modules.

→ In ~~the~~ next page (left)

The UML Module diagram, shows the dependency and relation between various modules in an software application.

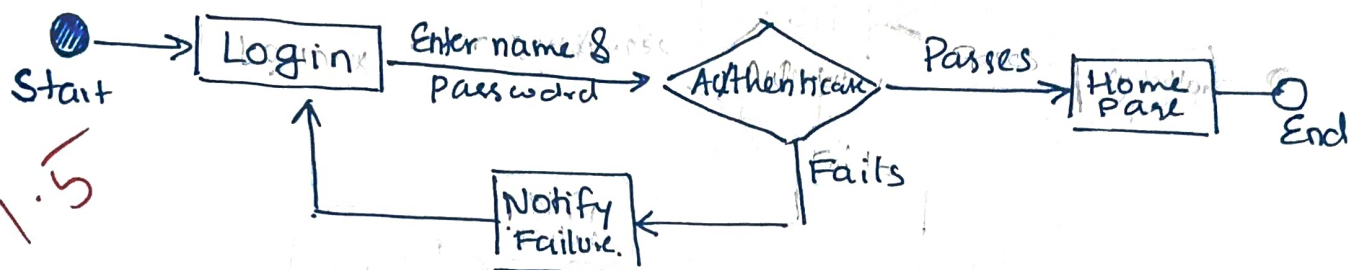
The two main rules.

- There should be no cyclical dependency
- - The higher level module should not depend on a lower level module.

Question 6 (1.5 marks)

What is the purpose of a UML activity diagram? Draw a small activity diagram for a login process with successful and unsuccessful outcomes.

The purpose of UML activity diagram, is to show the control flow how the software works from start to end.



Question 7 (1.5 marks)

Do you see a place where you would use the Adapter pattern in your group project? Explain.

Adapter pattern can be used to convert Legacy document in docx, or pdf ~~other file~~ to our ^{structured} Insight file format, which can be used for better queries and AI Insight efficient way.

Question 8 (1.5 marks)

Consider the following code:

```
public interface ILibraryUser
{
    void ReadInLibrary(Book book);
    void Borrow(Book book);
}
```

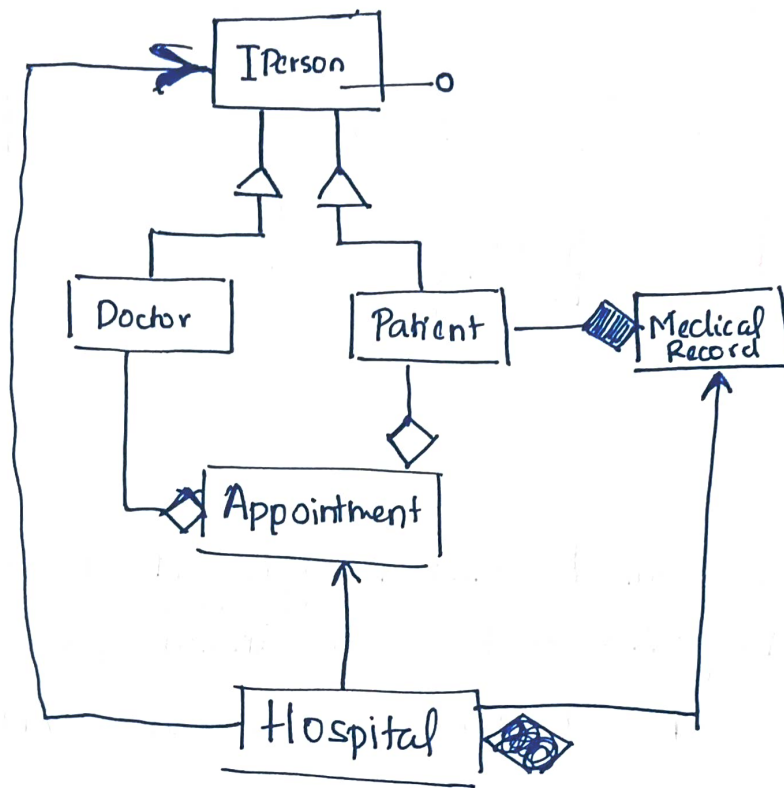
Your school library is open to the general public for reference, while students may also borrow books. Is any SOLID principle violated in the code above? If so, explain which one and why.

This fails Single Responsibility Principle, An object should only do one task, while the defined interface gives the responsibility to ReadInLibrary() and Borrow(). ~~which~~ These two methods needs to be separated for Single Responsibility Principle to become valid.

Question 9 (1 mark)

Draw a UML class diagram for Hospital, IPerson, Doctor, Patient, MedicalRecord, and Appointment. Show suitable type relationships.

Next page (left!)



Assumptions

- Medical record contains reports of ^{only} patients' prior medical history
- Hospital uses, IPerson, for having List of all doctor and patients in database. Similarly stores all appointments and medical records in separate list. Hence, using.

Question 10 (1 mark)

State the Liskov Substitution Principle. Give an example where a derived C# class violates it.

Liskov Substitution Principle: When a base class is replaced by a derived class object code should not break.

0.5
Eg: class Bird {
 void Eat();
 void Fly();
}

class Penguin: Bird {

// This fails because penguins don't fly.
(Hence you can't replace every Bird instance
 } with penguin and expect to work hence
 breaking code).

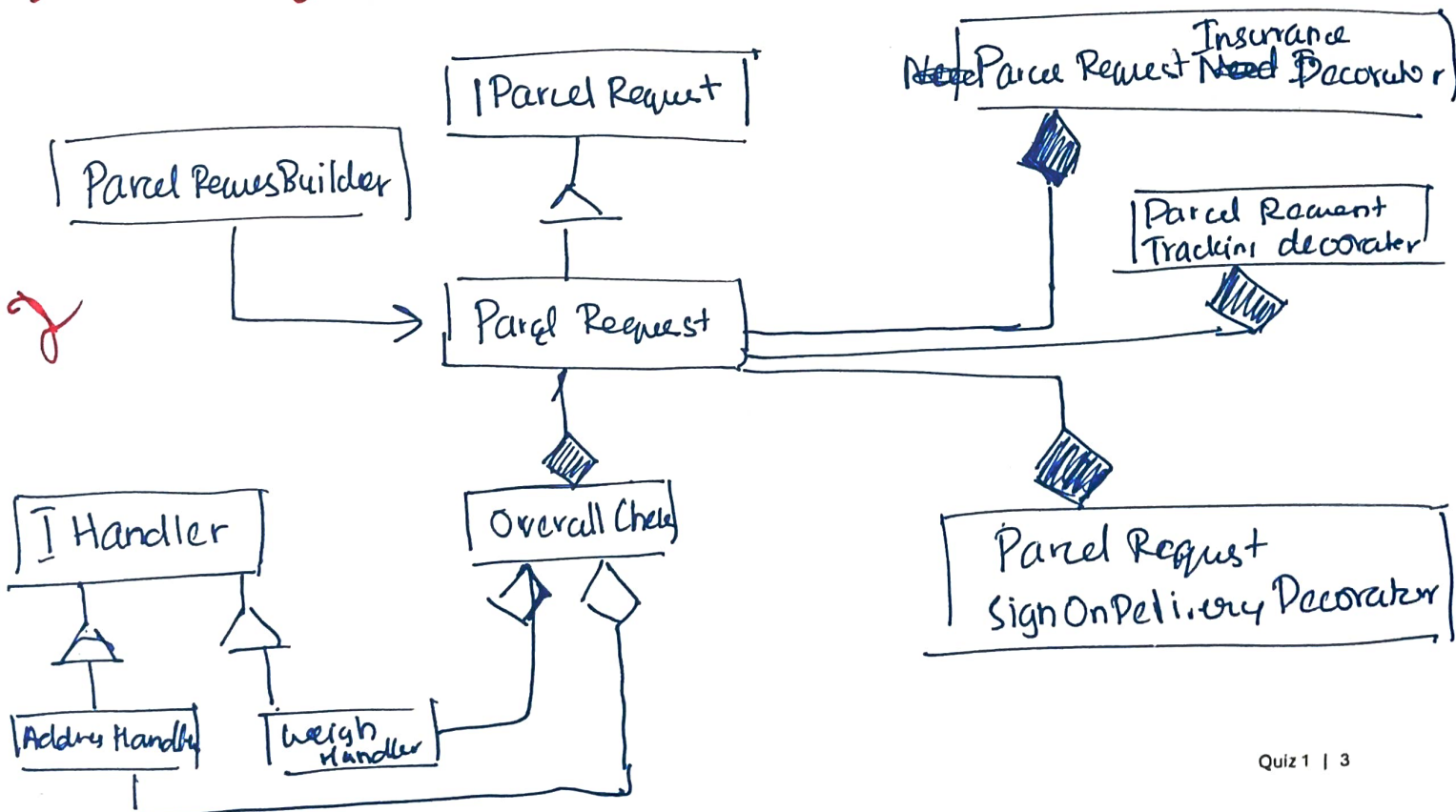
Question 11 (6 marks)

Design a parcel-dispatch system where:

1. A parcel request is assembled step by step with sender, recipient, weight, and destination.
2. Optional services such as insurance, tracking, and signature-on-delivery can be added dynamically.
3. The parcel passes through address, weight, and customs checks, stopping when a check fails.

Identify the three suitable design patterns, their category, mention which pattern addresses which request number, and draw a UML class diagram showing the principal types and relationships.

- 3
1. Builder (Creation design pattern).
 2. Decorator (Structural design pattern).
 3. Chain of Responsibility (Behavioural design pattern).



Question 12 (6 marks)

For the parcel-dispatch system above, write concise skeletal C# code demonstrating:

- Construction of one parcel request
- Addition of one optional service
- Processing through at least two checks

The pattern structure and interactions must be clear; method implementations may be abbreviated.

Construction of one parcel.

```
class ParcelRequestBuilder {
    private string sender;
    private string receipt;
    private string int weight;
    private string destination;

    void ParcelRequestBuilder Sendername( string sender)
    {
        this.sender = sender;
        return this;
    }

    void ParcelRequestBuilder Receipt( string receipt) {
        this.receipt = receipt;
        return this;
    }

    void ParcelRequestBuilder weight AdjustWeight( int weight) {
        this.weight = weight;
        return this;
    }

    void ParcelRequestBuilder Destination address( string address) {
        this. destination this.destination = destination;
        return this;
    }

    void IPParcelRequest Build() {
        return ParcelRequest( sender, receipt, weight, destination);
    }
}
```

// Usage.

```
var parcelrequest = ParcelRequestBuilder. Sender name ("Marafy")
    . Receipt ("receipt 0"). Adjust weight ( 3).
    Destination Address ("Palakad").
```

2. Addition of one additional service.

~~Parcel Request~~

~~Parcel Request~~
~~abstract class~~

interface IParcel Request {
public virtual bool Need Insurance(); ~~false~~;
public virtual bool Need Tracking(); ~~false~~;
public virtual bool Need Signature on delivery(); ~~false~~;
}

~~Parcel~~

class Parcel Request : IParcel Request {

~~private~~
public string sender;
public string receipt;
public int weight;
public string destination;

Parcel Request (string sender, string receipt, int weight, string destination) {

this.sender = sender;

this.receipt = receipt;

this.weight = weight;

this.destination = destination;

~~public override~~ Need Insurance() => false.

Need tracking() => false.

Need Signature on delivery() => false.

}

class

~~Parcel~~ Parcel Request Insurance Decorator : IParcel Request {
private Parcel Request pr;

Parcel Request Insurance Decorator (Parcel Request pr) {
this.pr = pr;

// Overwrite Need Insurance as true.
pr.Need Insurance() => True

} // Usage.

Parcel request pr; Parcel Request Insurance Decorator (pr);

Scratch Page / Overflow Answer Space

Use this page for rough work or to continue answers if you run out of space. Clearly label any extended answers.

3. ~~Ihandlers~~
~~abstract class~~
~~interface~~ Ihandler {
 public handler next handler ;
 public ^{void} handle ()
 }

class AddressHandler : Ihandler {
 // handles Address check it
 next handler . handle (^{new} weight handler)
 }

class WeightHandler : Ihandler {
 // checks weight and succeed.
 }

~~Overall check~~

class OverallCheck {
 private Parcel Request pr

}

}